

DANH SÁCH LIÊN KẾT (LINKED LIST)

ptbao@sgu.edu.vn

■ Đặc tính của CTDL mảng:

1. Ưu điểm

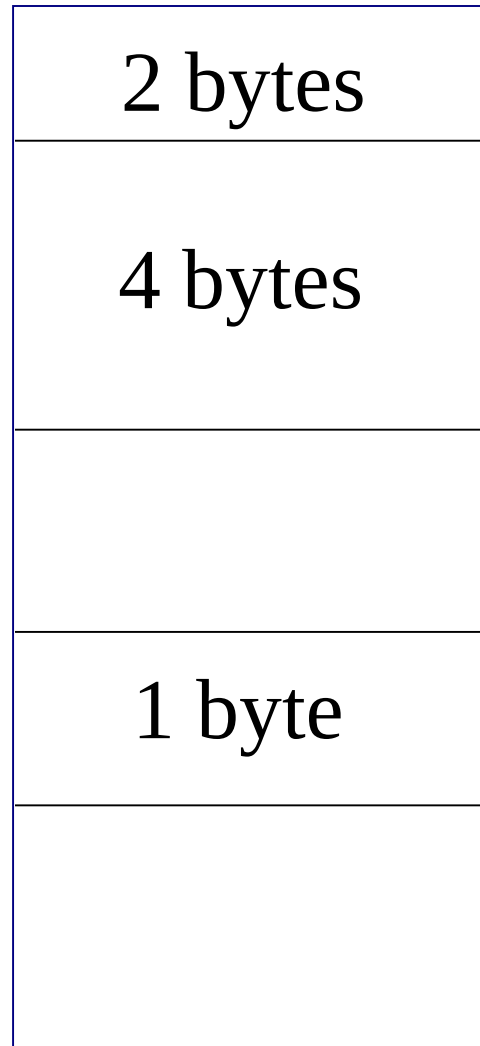
- Tốc độ xử lý cao do truy cập ngẫu nhiên.
- Dễ dàng cài đặt, sử dụng.

2. Khuyết điểm

- Cấp phát tĩnh, khi thì phí phạm bộ nhớ khi thì không đủ.
- Đòi hỏi vùng nhớ liên tục.

Mô hình cấp phát bộ nhớ

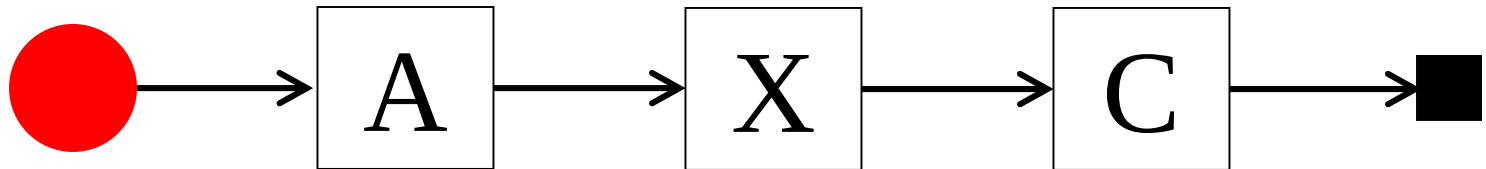
Địa chỉ 00F0



Cấu trúc dữ liệu DSLK

- Định nghĩa:

Là một CTDL gồm một dãy các phần tử (gọi là node) có cùng cấu trúc được liên kết với nhau tạo thành một chuỗi tuyến tính.



Cấu trúc dữ liệu DSLK (tt)

■ Tính chất:

1. DSLK sẽ có một số đặc tính như mảng 01 chiều. Đây là CTDL động, giúp tránh được nhược điểm về bộ nhớ.
2. Việc truy xuất dữ liệu thông qua phần tử đầu DSLK, và tuần tự từ phần tử này đến phần tử khác thông qua mối liên kết giữa các node. Node cuối cùng của DSLK sẽ trỏ đến một địa chỉ (giá trị) đặc biệt (với C là NULL). Tại sao phải trỏ đến 01 địa chỉ đặc biệt ?
3. DSLK là một CTDL tuyến tính, truy cập tuần tự, kích thước thay đổi theo nhu cầu.
4. Không cần vùng nhớ liên tục, hiệu quả ngay khi vùng nhớ bị phân mảnh.

Cấu trúc dữ liệu DSLK (tt)

- Các thao tác thường dùng:
 1. Khởi tạo DSLK
 2. Kiểm tra DSLK rỗng hay không ?
 3. Thêm một phần tử vào DSLK, thêm: vào đầu, cuối, giữa.
 4. Huỷ một phần tử ra khỏi DSLK, huỷ: ở đầu, cuối, giữa.
 5. Tìm một phần tử trong DSLK.
 6. Sắp xếp DSLK.

Cấu trúc dữ liệu DSLK (tt)

- Khai báo:

DSLK được định nghĩa như một CTDL đệ quy: mỗi phần tử gồm dữ liệu và một con trỏ trỏ đến địa chỉ của node kế tiếp.

- Quản lý:

Để quản lý một DSLK ta sẽ thông qua một phần tử đặc biệt là phần tử đầu tiên.

Từ phần tử đầu tiên này ta có thể truy cập bất cứ phần tử nào trong DSLK. Vì vậy dù làm gì ta sẽ không thể mất đầu phần tử đầu tiên nếu không ta sẽ không quản lý được DSLK.

Trong nhiều trường hợp ta cần xử lý nhiều ở cuối DSLK ta có thể thêm phần tử cuối DSLK để dễ quản lý.

DANH SÁCH LIÊN KẾT ĐƠN

Khai báo kiểu CTDL

- Kiểu cấu trúc trong C

```
typedef struct tagNode{
```

DataType Data;

// khai báo dữ liệu

```
struct tagNode *Next;
```

$$\};$$

```
typedef tagNode *Node
```

```
typedef struct tagList{
```

Node Head;

// phần tử đầu tiên

Node Tail;

// phần tử cuối, có thể không

// cần dùng cũng được

```
}LinkedList;
```

Khai báo kiểu CTDL (tt)

- Kiểu đối tượng trong C

```
class Node{  
    friend class LinkedList;  
    private:  
        DataType data;  
        Node *Next;  
    public:  
        Node(DataType d=giá trị đặc biệt ) {  
            data=d;Next=NULL;}  
        ~Node(){if(Next)delete Next;}  
};  
typedef Node *LinkedList;
```

```
class LinkedList{
protected:
    ListNode Head,Tail;
public:
    LinkedList(){Head=Tail=NULL;}
    // Vài constructor cần
    ~LinkedList(){if(Head)delete Head;}
    void insertNode(DataType);
    int searchNode(DataType);
    void deleteNode(DataType);
    void addTail(DataType);
    void printList();
    void addHead(DataType);
    void addAfter(DataType,DataType);
private:
    void addTail(ListNode);
    void addHead(ListNode);
    ListNode createNode(DataType);
    void insertNode(ListNode);
    int searchNode(ListNode);
    int deleteNode(DataType);
    int addAfter(ListNode,DataType);
};
```

Các thao tác

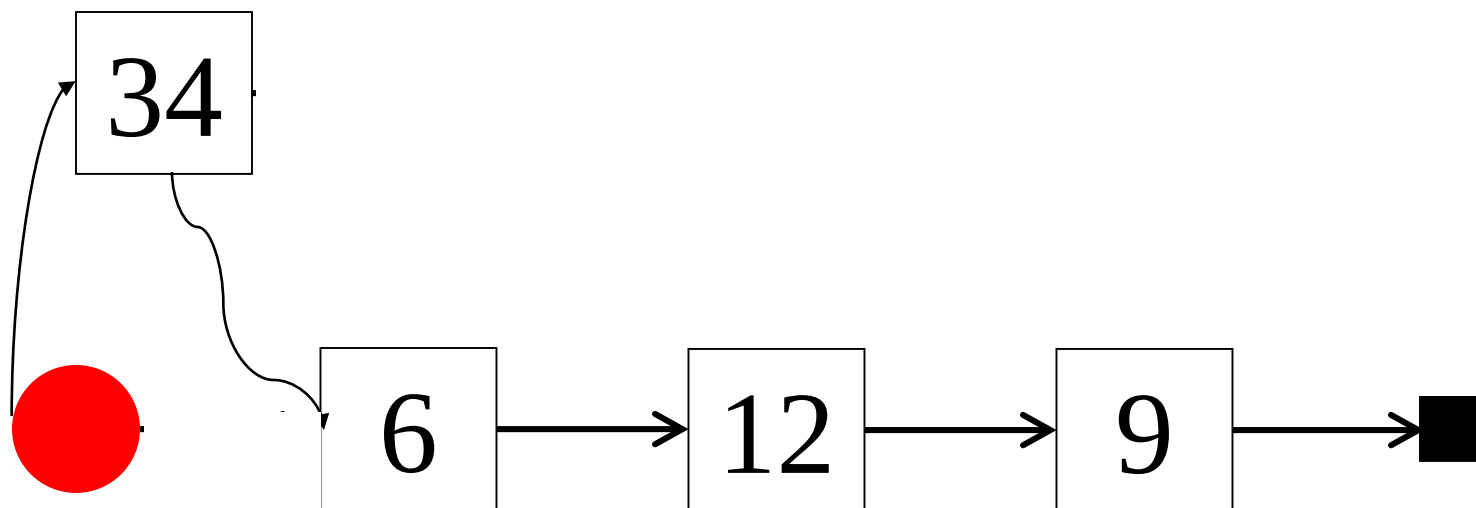
- Khởi tạo DSLK rỗng

```
void initLinkedList(LinkedList &l){  
    l.Head = l.Tail = NULL;  
}
```

- Kiểm tra DSLK rỗng ?

```
int isEmpty(LinkedList l){  
    return (l.Head==NULL);  
}
```

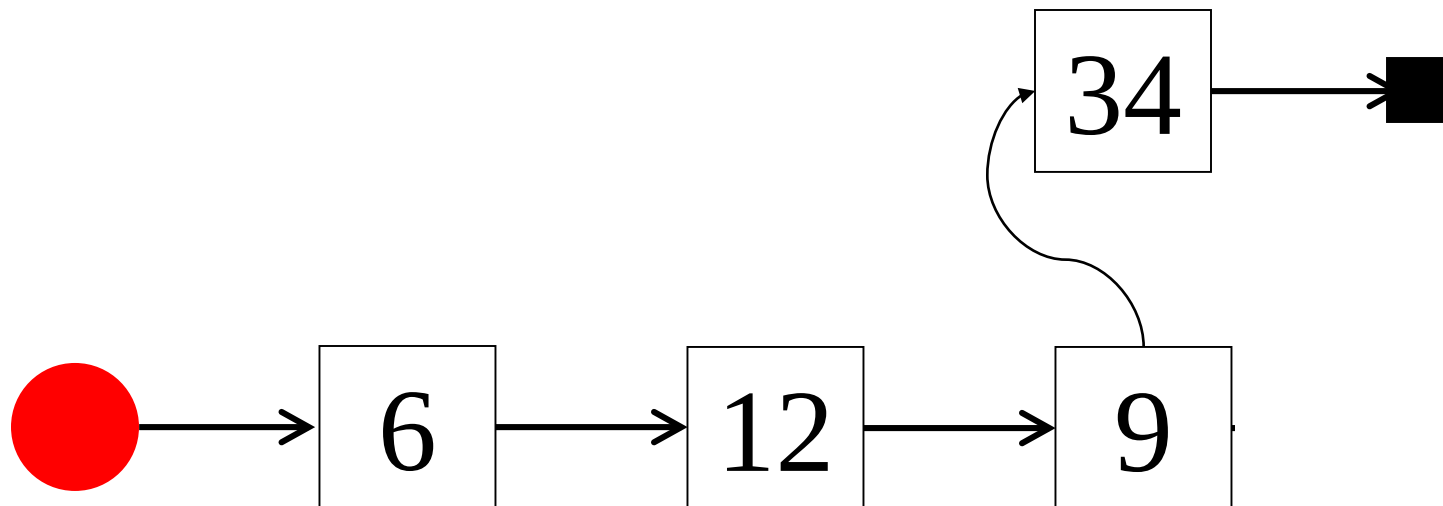
Thêm một phần tử vào đầu DSLK đơn



■ Thêm 01 phần tử vào đầu DSLK

```
int insertNode2Head(LinkedList &l, DataType x){
    Node p = new tagNode;
    if(p==NULL) return 0;
    p->Data = x;
    p->Next = NULL;
    if(isEmpty(l)){
        l.Head = l.Tail = p;
    }
    else {
        p->Next = l.Head;
        l.Head = p;
    }
    return 1;
}
```

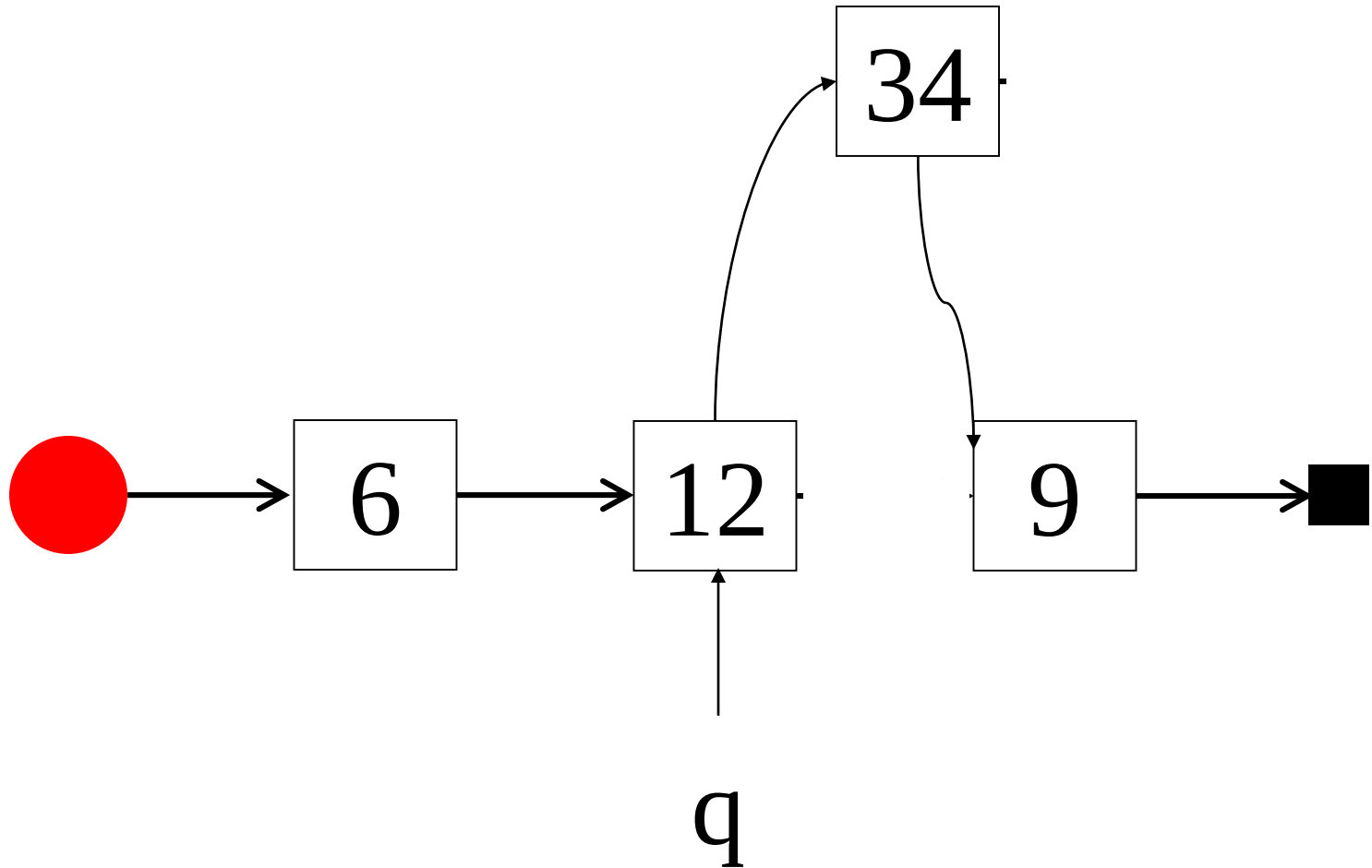
Thêm một phần tử vào cuối DSLK đơn



■ Thêm 01 phần tử vào cuối DSLK

```
int insertNode2Tail(LinkedList &l, DataType x){  
    Node p = new tagNode;  
    if(p==NULL)return 0;  
    p->Data = x;  
    p->Next = NULL;  
    if(isEmpty(l)){  
        l.Head = l.Tail = p;  
    }  
    else{  
        l.Tail->Next = p;  
        l.Tail = p;  
    }  
    return 1;  
}
```

Thêm một phần tử vào sau phần tử q của DSLK đơn



■ Thêm 01 phần tử vào sau node q trong DSLK

```
int insertNode2After(LinkedList &l, Node q, DataType x){
```

```
    Node p = new tagNode;
```

```
    if(p==NULL)return 0;
```

```
    p->Data = x;
```

```
    p->Next = NULL;
```

```
    if(isEmpty(l)){
```

```
        l.Head = l.Tail = p;
```

```
    }
```

```
    else{
```

```
        p->Next = q->Next;
```

```
        q->Next = p;
```

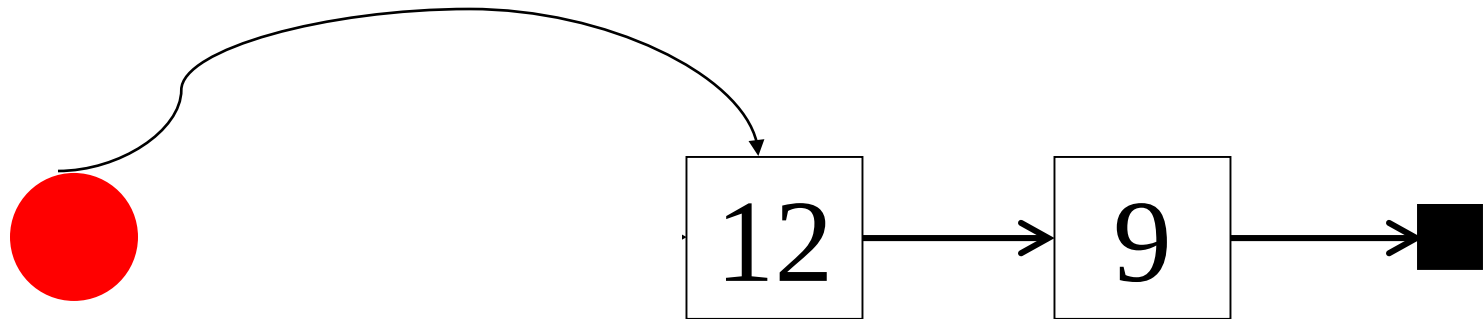
```
        if(q==l.Tail)l.Tail = p;
```

```
    }
```

```
    return 1;
```

```
}
```

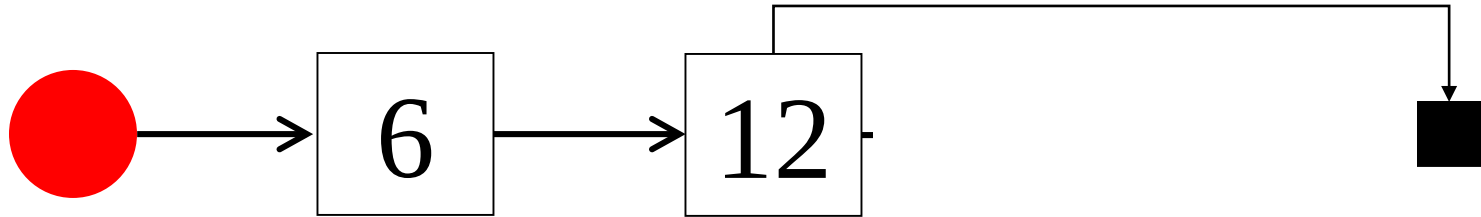
Hủy một phần tử ở đầu DSLK đơn



- Hủy 01 phần tử ở đầu DSLK

```
int removeNodeAtHead(LinkedList &l){  
    Node p = l.Head;  
    if(isEmpty(l)) return -1  
    l.Head = p->Next;  
    if(l.Head==NULL)l.Tail=NULL;  
    p->Next = NULL;  
    delete p;  
    return 1;  
}
```

Hủy một phần tử ở cuối DSLK đơn



■ Hủy 01 phần tử ở cuối DSLK

```
int removeNodeAtTail(LinkedList &l){
```

```
    Node p = l.Head;
```

```
    Node q = NULL;
```

```
    while(p!=l.Tail){
```

```
        q = p;
```

```
        p = p->Next;
```

```
    }
```

```
    if(!p) return -1;
```

```
    if(q){
```

```
        q->Next = NULL;
```

```
        l.Tail = q;
```

```
    }
```

```
    else {
```

```
        l.Head = l.Tail = NULL;
```

```
    }
```

```
    delete p;
```

```
    return 1;
```

```
}
```

■ Hủy 01 phần tử có giá trị X trong DSLK

```
int removeNode (LinkedList &l, DataType X){
    Node p = l.Head;
    if(!p) return -1;
    if(p->Data!=X && l.Tail->Data!=X){
        Node q= NULL;
        while(p->Data!=X && p){
            q = p; p = p->Next;
        }
        if(!p)return 0;
        q -> Next = p->Next;
        p->Next = NULL;
        delete p;
    }
    else{
        if(p->Data bằng X)return removeNodeAtHead(l);
        if(l.Tail->Data bằng X)return removeNodeAtTail(l);
    }
    return 1;
}
```

- Tìm 01 phần tử trong DSLK

```
Node searchNode(LinkedList l, DataType x){
```

```
    Node p = l.Head;
```

```
    if(p==NULL) return NULL;
```

```
    while (p){
```

```
        if(p->Data bằng x)return p;
```

```
        p = p->Next;
```

```
    }
```

```
    return NULL;
```

```
}
```

- Nối DSLK này vào cuối DSLK kia

```
void appendList(LinkedList &l, LinkedList &l1){  
    if(isEmpty(l))l = l1;  
    else{  
        l.Tail->Next = l1.Head;  
        if(!isEmpty(l1))l.Tail = l1.Tail;  
    }  
}
```

- Sắp xếp DSLK theo PP MergeSort

```
void mergeSort(LinkedList &l){  
    LinkedList l1,l2;  
    if(l.Head==l.Tail)return;  
    initLinkedList(l1);  
    initLinkedList(l2);  
    divideList(l,l1,l2);  
    mergeSort(l1);  
    mergeSort(l2);  
    mergeList(l,l1,l2);  
}
```

■ Sắp xếp DSLK theo PP MergeSort (tt)

```
void divideList(LinkedList &l,LinkedList &l1,LinkedList &l2){
    Node p;
    while(!isEmpty(l)){
        p = l.Head;
        l.Head = p->Next;
        p->Next = NULL;
        insertNodeAtTail(l1,p);
        if(!isEmpty(l)){
            p = l.Head;
            l.Head = p->Next;
            p->Next = NULL;
            insertNodeAtTail(l2,p);
        }
    }
    initLinkedList(l);
}
```

■ Sắp xếp DSLK theo PP MergeSort (tt)

```
void mergeList(LinkedList &l,LinkedList &l1,LinkedList &l2){
    Node p;
    while(!isEmpty(l1)&&!isEmpty(l2)){
        if(l1.Head->Data nhỏ hơn bằng l2.Head->Data){
            p = l1.Head;
            l1.Head = p->Next;
            p->Next = NULL;
        }
        else{
            p = l2.Head;
            l2.Head = p->Next;
            p->Next = NULL;
        }
        insertNodeAtTail(l,p);
    }
    if(!isEmpty(l1))appendList(l,l1);
    if(!isEmpty(l2))appendList(l,l2);
    initLinkedList(l1);
    initLinkedList(l2);
}
```

- Sắp xếp DSLK theo PP QuickSort

```
void quickSort(LinkedList &l){
```

```
    LinkedList l1,l2;
```

```
    if(l.Head==l.Tail)return;
```

```
    initLinkedList(l1);
```

```
    initLinkedList(l2);
```

```
    splitList(l,l1,l2,X);
```

```
    quickSort(l1);
```

```
    quickSort(l2);
```

```
    concatList(l,l1,X,l2);
```

```
    initLinkedList(l1);
```

```
    initLinkedList(l2);
```

```
}
```

■ Sắp xếp DSLK theo PP QuickSort (tt)

```
void splitList(LinkedList &l, LinkedList &l1, LinkedList &l2, Node &x){  
    Node p;  
    x = l.Head;  
    l.Head = x->Next;  
    x->Next = NULL;  
    while(!isEmpty(l)){  
        p = l.Head;  
        l.Head = p->Next;  
        p->Next = NULL;  
        if(p->Data nhỏ hơn bằng x->Data)insertNodeAtTail(l1,p);  
        else insertNodeAtTail(l2,p);  
    }  
    initLinkedList(l);  
}
```

■ Sắp xếp DSLK theo PP QuickSort (tt)

```
void concatList(LinkedList &l, LinkedList &l1, Node &x, LinkedList  
    &l2){  
    appendList(l, l1);  
    insertNodeAtTail(l, x);  
    appendList(l, l2);  
    initLinkedList(l1);  
    initLinkedList(l2);  
}
```

■ Sắp xếp DSLK theo PP RadixSort

```
void radixSort(LinkedList &l){  
    LinkedList temp[10];  
    if(l.Head==l.Tail)return;  
    for(int i=0;i<10;i++)initLinkedList(temp[i]);  
    int numDigit = getMaxDigit(l);  
    for(i=0;i<numDigit;i++){  
        send2Box(i,l,temp);  
        for(int j=0;j<10;j++){  
            if(!isEmpty(temp[j]))appendList(l,temp[j]);  
            initLinkedList(temp[j]);  
        }  
    }  
}
```

- Sắp xếp DSLK theo PP RadixSort (tt)

```
void send2Box(int n, LinkedList &l, LinkedList  
temp[]){
```

```
Node p;
```

```
while(!isEmpty(l)){
```

```
    p = getHead(l);
```

```
    addTail(temp[getDigit(n,p->Data)],p);
```

```
}
```

```
initLinkedList(l);
```

```
}
```

- Sắp xếp DSLK theo PP RadixSort (tt)

```
int getDigit(int n, int t){  
    long temp=1;  
    for(int i=0;i<n;i++)temp*=10; (t=t/10)  
    return (t/temp)%10; (t%10)  
}
```

```
int getMaxDigit(l){  
    int max = findMax(l);  
    return countDigit(max);  
}
```

- Sắp xếp DSLK theo PP RadixSort (tt)

```
int findMax(LinkedList l){
```

```
}
```

```
int countDigit(int m){
```

```
    int count = 0;
```

```
    while(m){
```

```
        m/=10;
```

```
        count++;
```

```
    }
```

```
    return count;
```

```
}
```

NGĂN XẾP (STACK)

CTDL Ngăn xếp

- Định nghĩa:

là một vật chứa (container) các đối tượng làm việc theo cơ chế **LIFO** (**L**ast **I**n **F**irst **O**ut), nghĩa là đối tượng được đưa vào sau sẽ được lấy ra trước.

CTDL Ngăn xếp (tt)

- Tính chất:

1. Các đối tượng được thêm vào bất kỳ lúc nào nhưng chỉ có đối tượng được thêm sau cùng mới được phép lấy ra.
2. Thao tác thêm 01 phần tử được gọi là “Push”, thao tác lấy 01 phần tử ra là “Pop”.

CTDL Ngăn xếp (tt)

- Ví dụ:

Băng đạn súng AK, tháp Hà Nội, ...

- Ứng dụng:

Khử đệ quy, tổ chức các quá trình tìm kiếm theo chiều sâu và quay lui, vết cặn, tính toán biểu thức, ...

CTDL Ngăn xếp (tt)

- Định nghĩa

Stack là một CTDL ADT hỗ trợ 02 thao tác chính:

1. push(o): thêm đối tượng o vào đầu stack
2. pop(): lấy đối tượng ở đầu stack ra.

- Thao tác hỗ trợ:

- 1.size(): số phần tử trong stack
- 2.isEmpty(): kiểm tra stack rỗng
- 3.top(): trả giá trị phần tử ở đầu stack

CTDL Ngăn xếp (tt)

- CTDL

1. Mảng
2. DSLK

- Cài đặt

1. Mảng 01 chiều: khai báo 01 mảng 01 chiều có kích thước N phần tử, và các phần tử được đánh số từ 0 đến $N-1$.

CTDL Ngăn xếp (tt)

2. DSLK đơn: DSLK có những đặc tính phù hợp để làm stack

STT	Stack	DSLKD
1	push	insertHead
2	pop	removeHead
3	size	getCount
4	isEmpty	isEmpty
5	top	getHead

CTDL Ngăn xếp (tt)

```
typedef LinkedList Stack;  
int isEmpty(Stack st){  
    return st.Head==NULL;  
}  
int size(Stack st){  
    Node p = st.Head;  
    int count = 0;  
    while(p){  
        count++;  
        p = p->Next;  
    }  
    return count;  
}
```

CTDL Ngăn xếp (tt)

```
DataType top(Stack st){  
    return st.Head->Data;  
}  
  
int push(Stack st, DataType x){  
    // chèn vào đầu DSLK st  
}  
  
DataType pop(Stack st){  
    // Lấy phần tử đầu DSLK st  
}
```

THUẬT TOÁN BA LAN NGƯỢC

Thuật toán Ba Lan ngược (Reverse Polish Notation - RPN)

Bài toán:

$$(1+5)*(12 - (8+1))$$

Biến đổi $\longrightarrow$ 1 5 + 12 8 1 + - *

Thuật toán chuyển đổi:

1. Khởi tạo Stack rỗng (chứa các phép toán).

2. Lặp cho đến khi kết thúc biểu thức:

- Đọc 01 phần tử của biểu thức (01 phần tử có thể là hằng, biến, phép toán, “)” hay “(”).
- Nếu phần tử là:
 - ✓ “(”: đưa vào Stack.
 - ✓ “)”: lấy các phần tử của Stack ra cho đến khi gặp “(” trong Stack.
 - ✓ Một phép toán:
 - ❖ Nếu Stack rỗng: đưa vào Stack.
 - ❖ Nếu Stack khác rỗng và phép toán có độ ưu tiên cao hơn phần tử ở đầu Stack: đưa vào Stack.
 - ❖ Nếu Stack khác rỗng và phép toán có độ ưu tiên thấp hơn hoặc bằng phần tử ở đầu Stack: lấy phần tử từ Stack ra; sau đó lặp lại việc so sánh với phần tử ở đầu Stack.
 - ✓ Hằng hoặc biến: Lấy ra.

3. Lấy hết tất cả các phần tử của Stack ra.

(Ta xem “(” có độ ưu tiên thấp hơn độ ưu tiên của các phép toán)

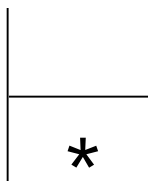
Ví dụ

$$7 * 8 - (2 + 3)$$



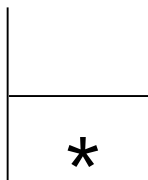
7

$$* 8 - (2 + 3)$$



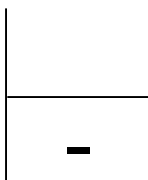
7

$$8 - (2 + 3)$$



7 8

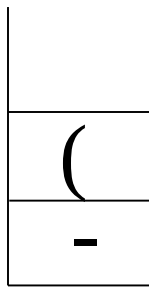
$$- (2 + 3)$$



7 8 *

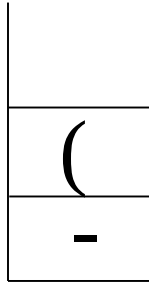
Lấy * ra, đưa - vào

(2+3)



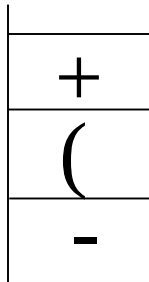
7 8 *

2+3)



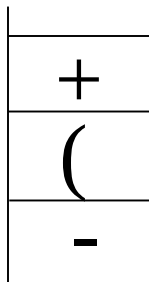
7 8 * 2

+3)



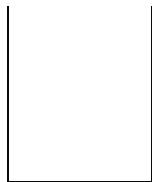
7 8 * 2

3)



7 8 * 2 3

)



Lấy + (- ra

7 8 * 2 3 + -

$7 * 8 - (2 + 3)$

Kết Quả



7 8 * 2 3 + -

Thuật toán tính giá trị biểu thức Ba lan ngược

7 8 * 2 3 + -

Kết Quả



51

Thuật toán tính giá trị:

1. Khởi tạo Stack rỗng (chứa hằng hoặc biến).
2. Lặp cho đến khi kết thúc biểu thức:
 - Đọc 01 phần tử của biểu thức (01 phần tử có thể là hằng, biến, phép toán).
 - Nếu phần tử là hằng hay biến: đưa vào Stack.
 - Ngược lại:
 - ✓ Lấy ra 02 phần tử của Stack.
 - ✓ Áp dụng phép toán cho 02 phần tử vừa lấy ra.
 - ✓ Đưa kết quả vào Stack.
3. Giá trị của biểu thức chính là phần tử cuối cùng của Stack.

Ví dụ

7 8 * 2 3 + -

7

8 * 2 3 + -

8
7

*2 3 + -

56

Thực hiện phép toán *

2 3 + -

2
56

3 + -

3
2
56

+ -

5
56

Thực hiện phép toán +

-

51

Thực hiện phép toán -

7 8 * 2 3 + -

Kết Quả



51

HÀNG ĐỢI (QUEUE)

CTDL Hàng đợi

- Định nghĩa:

Là một vật chứa (Container) các đối tượng làm việc theo cơ chế **FIFO** (**F**irst **I**n **F**irst **O**ut), nghĩa là phần tử nào vào trước sẽ là phần tử ra trước.

CTDL Hàng đợi (tt)

- Tính chất:
 1. Các đối tượng được thêm vào bất kỳ lúc nào ở cuối nhưng chỉ có đối tượng được thêm vào đầu tiên mới được phép lấy ra.
 2. Thao tác thêm 01 phần tử được gọi là “enqueue”, thao tác lấy 01 phần tử ra là “dequeue”.

CTDL Hàng đợi (tt)

- Ví dụ:

Việc xếp hàng mua vé, quản lý và phân phối tiến trình trong hệ điều hành, ...

- Ứng dụng:

Khử đệ quy, tổ chức các quá trình tìm kiếm theo chiều rộng và quay lui, vét cạn, tổ chức quản lý và phân phối tiến trình trong các hệ điều hành, tổ chức bộ đệm của bàn phím, ...

CTDL Hàng đợi (tt)

- Định nghĩa

Queue là một CTDL ADT hỗ trợ 02 thao tác chính:

1. enqueue(o): thêm đối tượng o vào cuối queue
2. dequeue(): lấy đối tượng ở đầu queue ra.

- Thao tác hỗ trợ:

- 1.size(): số phần tử trong queue
- 2.isEmpty(): kiểm tra queue rỗng
- 3.front(): trả giá trị phần tử ở đầu queue

CTDL Hàng đợi (tt)

- CTDL

1. Mảng
2. DSLK

- Cài đặt

1. Mảng 01 chiều: khai báo 01 mảng 01 chiều có kích thước N phần tử, và các phần tử được đánh số từ 0 đến $N-1$.

CTDL Hàng đợi (tt)

2. DSLK đơn: DSLK có những đặc tính phù hợp để làm stack

STT	Queue	DSLKD
1	enqueue	insertTail
2	dequeue	removeHead
3	size	getCount
4	isEmpty	isEmpty
5	front	getHead

CTDL Hàng đợi (tt)

```
typedef LinkedList Queue;
int isEmpty(Queue qu){
    return qu.Head==NULL;
}
int size(Queue qu){
    Node p = qu.Head;
    int count = 0;
    while(p){
        count++;
        p = p->Next;
    }
    return count;
}
```

CTDL Hàng đợi (tt)

```
DataType front(Queue qu){  
    return qu.Head->Data;  
}
```

```
int enqueue (Queue qu, DataType x){  
    // chèn vào cuối DSLK qu  
}
```

```
DataType dequeue (Queue qu){  
    // Lấy phần tử đầu DSLK qu  
}
```

DSLK CÓ THỨ TỰ
(ORDERED LIST)

CTDL DSLK Có thứ tự

- Định nghĩa:

Là một vật chứa (Container) các đối tượng theo một trình tự nhất định. Khi thêm phần tử ta phải đảm bảo quan hệ thứ tự không thay đổi.

DSLK VÒNG

CTDL DSLK Đơn vòng

- Định nghĩa:

Là một DSLK mà phần tử kế tiếp của phần tử cuối DSLK chính là phần tử đầu DSLK.

DSLK Đơn vòng (tt)

- Thêm ở đầu
 - Như bình thường
 - $l.Tail \rightarrow Next = l.Head$
- Thêm ở cuối
 - Như bình thường
 - $l.Tail \rightarrow Next = l.Head$
- Thêm ở giữa
 - Như bình thường

DSLK Đơn vòng (tt)

- Hủy ở đầu
 - Như bình thường
 - $l.Tail \rightarrow Next = l.Head$
- Hủy ở cuối
 - Như bình thường
 - $l.Tail \rightarrow Next = l.Head$
- Hủy ở giữa
 - Như bình thường

DSLK Đơn vòng (tt)

- Duyệt

```
p = l.Head;
```

```
do{
```

```
    // xử lý, ví dụ in giá trị các node: printf(“%d”,p->Data);
```

```
    p = p->Next;
```

```
}while(p!=l.Head);
```

DSLK ĐÔI

(DOUBLE LINKED LIST)

DSLK Đôi

- Định nghĩa:

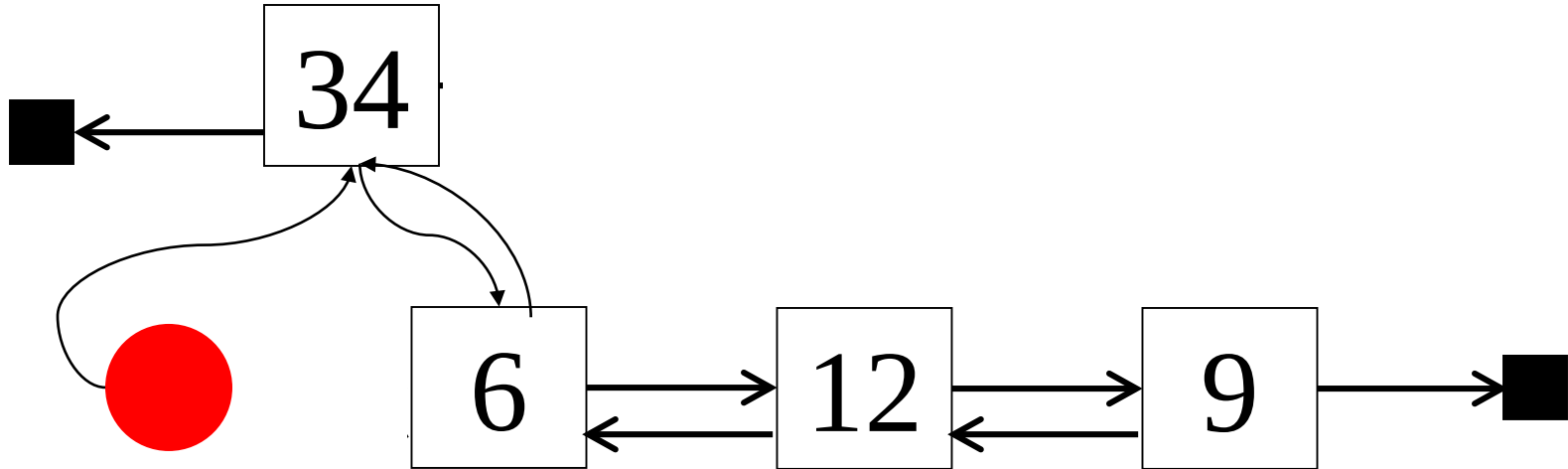
Là DSLK mà mỗi phần tử sẽ có 02 mối liên kết.

- Khai báo:

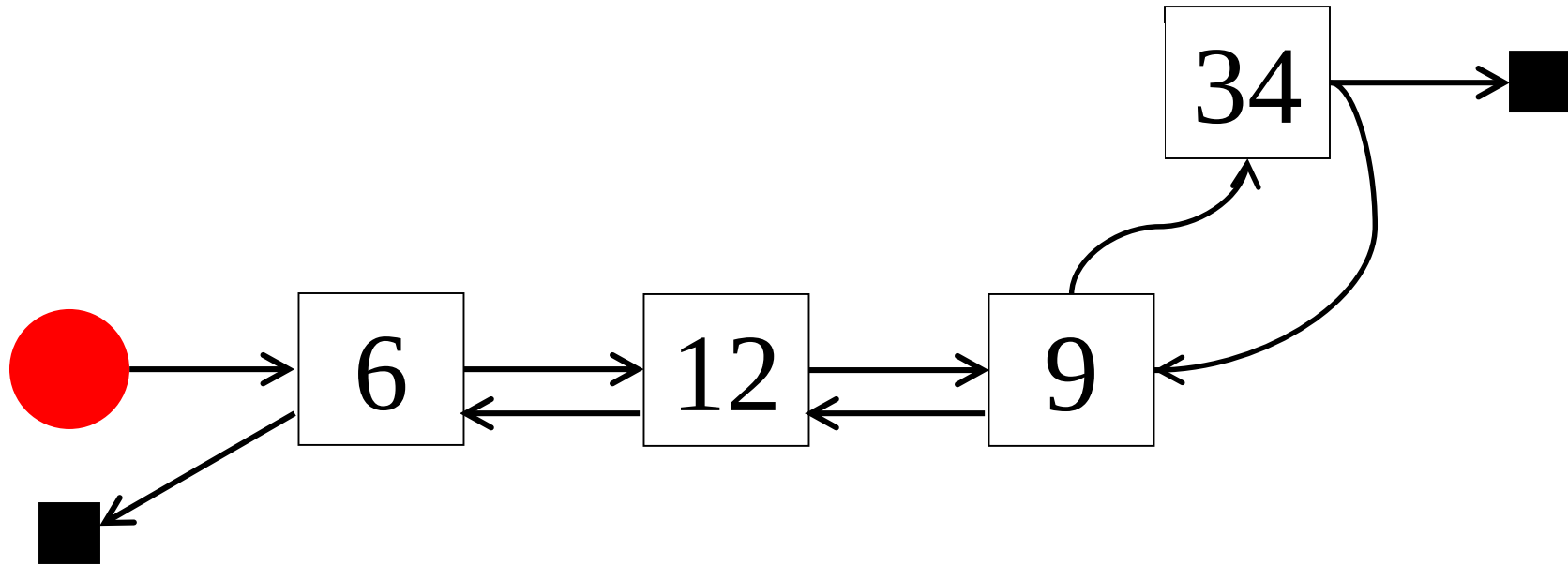
```
typedef struct tagNode{  
    DataType Data;  
    tagNode *Next;  
    tagNode *Prev;  
};
```

```
typedef struct tagNode{  
    DataType Data;  
    tagNode *Left;  
    tagNode *Right;  
};
```

Thêm một phần tử vào đầu DSLK đôi



Thêm một phần tử vào cuối DSLK đôi



DSLK Đôi (tt)

- Thêm ở đầu

$p \rightarrow \text{Next} = l.\text{Head};$

$l.\text{Head} \rightarrow \text{Prev} = p;$

$l.\text{Head} = p;$

- Thêm ở cuối

$l.\text{Tail} \rightarrow \text{Next} = p;$

$p \rightarrow \text{Prev} = l.\text{Tail};$

$l.\text{Tail} = p;$

DSLK Đôi (tt)

- Thêm p ở sau phần tử q

$p \rightarrow \text{Next} = q \rightarrow \text{Next};$

$p \rightarrow \text{Prev} = q;$

$q \rightarrow \text{Next} \rightarrow \text{Prev} = p;$

$q \rightarrow \text{Next} = p;$

DSLK Đôi (tt)

- Hủy ở đầu

```
p = l.Head;
```

```
l.Head = p->Next;
```

```
l.Head->Prev = NULL;
```

```
p->Next = NULL;
```

```
delete p;
```

DSLK Đôi (tt)

- Hủy ở cuối

$p = l.Tail;$

Tìm q trước p

$l.Tail = q;$

$l.Tail \rightarrow Next = NULL;$

$p \rightarrow Next = p \rightarrow Prev = NULL;$

delete p;

DSLK Đôi (tt)

- Hủy p ở sau q
 $q \rightarrow \text{Next} = p \rightarrow \text{Next};$
 $p \rightarrow \text{Next} \rightarrow \text{Prev} = q;$
 $p \rightarrow \text{Next} = p \rightarrow \text{Prev} = \text{NULL};$
 delete p;

DANH SÁCH LIÊN KẾT ĐA